

Deployment & High Availability Guideline

Greenfields IMS — MVP Audit & Incident Logs

Technical Assignment untuk PT. Greenfields Indonesia

Disusun oleh: Fauzie Ikhlasul Amnur

24 May 2026

Table of Contents

Contents

Table of Contents.....	2
1. Ringkasan Deployment.....	4
2. Kebutuhan Infrastruktur.....	4
3. Komponen Aplikasi	4
4. Arsitektur Deployment.....	4
5. Langkah-Langkah Deployment.....	5
5.1 Install System Dependencies	5
5.2 Setup PostgreSQL.....	5
5.3 Deployment Backend.....	6
5.4 Deployment Frontend	6
5.5 Konfigurasi systemd Service	6
5.6 Konfigurasi Nginx Reverse Proxy	7
6. Post-Deployment Verification	7
6.1 Health Check.....	7
6.2 Test Login.....	7
6.3 Test Frontend	7
7. Strategi High Availability	7
7.1 Process Management	7
7.2 Horizontal Scaling	8
7.3 Vertical Scaling.....	8
7.4 Zero-Downtime / Near-Zero-Downtime Deployment	8
8. Backup dan Restore	9
8.1 Backup PostgreSQL.....	9
8.2 Restore PostgreSQL.....	9
9. Monitoring dan Logging.....	9
10. Catatan Keamanan	10

11. Strategi Rollback.....	10
12. Kesimpulan	11

1. Ringkasan Deployment

Dokumen ini menjelaskan guideline deployment dan high availability untuk Greenfields IMS (Incident Management System). Environment yang direkomendasikan menggunakan Ubuntu 22.04 LTS sebagai sistem operasi server, PostgreSQL 14+ sebagai database, FastAPI (Python 3.12+) sebagai backend framework, React 19 sebagai frontend, Nginx sebagai reverse proxy dan static file server, serta systemd sebagai process manager.

Arsitektur deployment dirancang agar portable (dapat di-deploy di VM atau VPS), scalable (mendukung horizontal scaling melalui Nginx upstream), dan siap diintegrasikan ke lingkungan produksi dengan strategi high availability dan near-zero downtime deployment.

2. Kebutuhan Infrastruktur

Komponen	Spesifikasi Minimum	Spesifikasi Rekomendasi
CPU	1 Core	2 Core
RAM	2 GB	4 GB
Storage	20 GB SSD	50 GB SSD
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Database	PostgreSQL 14+	PostgreSQL 14+
Python	3.12+	3.12+
Node.js	20 LTS+	20 LTS+

Catatan: Spesifikasi minimum cukup untuk menjalankan sistem dalam mode development/testing dengan beban rendah. Spesifikasi rekomendasi diperlukan untuk production dengan ribuan insiden aktif.

3. Komponen Aplikasi

Layer	Teknologi	Fungsi
Web Server	Nginx	Reverse proxy, serve static files, rate limiting
Application Server	Uvicorn (2 workers)	ASGI server untuk FastAPI
Backend Framework	FastAPI 0.104	REST API, autentikasi, business logic
Frontend	React 19 + Vite 8	SPA user interface
Database	PostgreSQL 14+	Data storage, audit trail
Process Manager	systemd	Auto-restart, process hardening

4. Arsitektur Deployment

Internet / Browser

|
v

Nginx Reverse Proxy (Port 80)

```
|-- Serve React Static Files (/opt/greenfields/frontend/dist)
|-- Proxy /api/* ke http://127.0.0.1:8000
|-- Rate limiting: 5 req/menit/IP untuk /api/auth/login
|-- Health check: /health -> http://127.0.0.1:8000/health
```

```
|
v
```

Uvicorn (localhost:8000, 2 workers)

```
|
v
```

FastAPI Backend

```
|-- JWT Auth (HS256, bcrypt)
|-- psycpg2 ThreadedConnectionPool (min 2, max 8)
|-- Statement timeout: 15 detik
```

```
|
v
```

PostgreSQL (localhost:5432)

```
Database: greenfields_audit
```

Seluruh komponen berjalan dalam satu VM untuk deployment MVP. Pemisahan database server dapat dilakukan saat scaling.

5. Langkah-Langkah Deployment

5.1 Install System Dependencies

Jalankan perintah berikut untuk menginstall dependencies sistem:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y nginx postgresql postgresql-contrib python3 python3-pip python3-venv curl git
```

Install Node.js 20 LTS untuk build frontend:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
```

5.2 Setup PostgreSQL

```
sudo systemctl enable postgresql
sudo systemctl start postgresql

# Create database
sudo -u postgres psql -c "CREATE DATABASE greenfields_audit;"
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres';"

# Run migrations
```

```
for f in /opt/greenfields/backend/app/migration/*.sql; do
    sudo -u postgres psql -d greenfields_audit -f "$f"
done
```

5.3 Deployment Backend

```
# Create system user
sudo useradd -r -s /bin/false -m -d /opt/greenfields greenfields

# Copy source code to /opt/greenfields/backend
sudo mkdir -p /opt/greenfields/backend
sudo cp -r backend/* /opt/greenfields/backend/
sudo chown -R greenfields:greenfields /opt/greenfields

# Setup Python virtual environment
sudo -u greenfields python3 -m venv /opt/greenfields/backend/.venv
sudo -u greenfields /opt/greenfields/backend/.venv/bin/pip install -r /opt/greenfields/backend/requirements.txt
```

Buat file .env backend:

```
# /opt/greenfields/backend/.env
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/greenfields_audit
SECRET_KEY=<ganti-dengan-random-secret-yang-kuat>
ACCESS_TOKEN_EXPIRE_MINUTES=480
```

5.4 Deployment Frontend

```
# Copy source frontend
sudo mkdir -p /opt/greenfields/frontend
sudo cp -r frontend/* /opt/greenfields/frontend/
sudo chown -R greenfields:greenfields /opt/greenfields/frontend

# Build
cd /opt/greenfields/frontend
npm ci
npm run build

# Output: /opt/greenfields/frontend/dist/
```

5.5 Konfigurasi systemd Service

Backend dikelola menggunakan systemd service greenfields-backend.service. Konfigurasi ini menyediakan:

Automatic startup saat server boot (enable)

Auto-restart ketika service crash (Restart=always)

Graceful shutdown dengan SIGTERM (TimeoutStopSec=30)

Resource limits dan isolasi proses dasar (NoNewPrivileges, PrivateTmp, ProtectSystem)

```
sudo cp infra/systemd/greenfields-backend.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable greenfields-backend
sudo systemctl start greenfields-backend
sudo systemctl status greenfields-backend
```

5.6 Konfigurasi Nginx Reverse Proxy

Nginx bertanggung jawab untuk menyajikan frontend static files, meneruskan request /api ke backend, menyediakan endpoint /health, dan mendukung scaling melalui upstream backend.

```
sudo cp infra/nginx/greenfields.conf /etc/nginx/sites-available/greenfields
sudo ln -sf /etc/nginx/sites-available/greenfields /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
```

6. Post-Deployment Verification

6.1 Health Check

```
curl http://localhost/health
```

Expected response:

```
{
  "status": "healthy",
  "app": "Greenfields Audit System",
  "database": "connected"
}
```

6.2 Test Login

```
curl -X POST http://localhost/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"admin123"}'
```

Expected: JWT token dalam response

6.3 Test Frontend

Buka <http://localhost> melalui browser. Halaman login harus tampil. Login dengan akun admin/admin123, maka dashboard akan dimuat dan seluruh fitur (create/edit incident, manage users, audit log) tersedia sesuai role user.

7. Strategi High Availability

7.1 Process Management

systemd menjaga availability backend melalui auto-restart dan graceful shutdown:

```
[Service]
Restart=always
RestartSec=5
KillSignal=SIGTERM
TimeoutStopSec=30
```

Jika backend crash atau hang, systemd akan otomatis merestart dalam 5 detik. Saat shutdown, systemd mengirim SIGTERM dan memberikan waktu 30 detik untuk graceful shutdown (menyelesaikan request aktif).

7.2 Horizontal Scaling

Nginx upstream dapat diperluas dengan beberapa backend instance. Backend bersifat stateless karena authentication menggunakan JWT, sehingga request dapat didistribusikan ke beberapa instance tanpa sessions synchronization.

```
upstream greenfields_backend {
    least_conn;
    server 127.0.0.1:8000 max_fails=3 fail_timeout=30s;
    server 127.0.0.1:8001 max_fails=3 fail_timeout=30s;
    server 127.0.0.1:8002 max_fails=3 fail_timeout=30s;
}
```

Konfigurasi di atas menggunakan algoritma least_conn untuk mendistribusikan request ke backend dengan koneksi paling sedikit. Setiap backend instance berjalan pada port berbeda dan harus memiliki akses ke database yang sama.

7.3 Vertical Scaling

Meningkatkan resource VM dari 1 CPU / 2 GB RAM menjadi 2 CPU / 4 GB RAM atau lebih

Menambah jumlah Uvicorn workers sesuai kapasitas CPU (umumnya 2 worker per core)

Meningkatkan ukuran PostgreSQL connection pool (DB_POOL_MIN dan DB_POOL_MAX)

Memindahkan PostgreSQL ke dedicated database server jika traffic meningkat

7.4 Zero-Downtime / Near-Zero-Downtime Deployment

Strategi rolling deployment untuk update aplikasi tanpa downtime signifikan:

```
# 1. Build backend versi baru
# 2. Jalankan backend instance baru pada port 8001
uvicorn app.main:app --host 127.0.0.1 --port 8001

# 3. Lakukan health check pada instance baru
curl http://127.0.0.1:8001/health

# 4. Tambahkan instance baru ke Nginx upstream (update konfigurasi)
```

```
# 5. Reload Nginx tanpa memutus koneksi aktif
sudo nginx -s reload

# 6. Setelah traffic berpindah, hentikan instance lama
# 7. Restart instance lama dengan versi baru pada port 8000
# 8. Update Nginx upstream untuk menggunakan instance yang sudah diupdate
```

8. Backup dan Restore

8.1 Backup PostgreSQL

Jadwalkan backup harian menggunakan cron:

```
# /etc/cron.daily/greenfields-backup
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR=/backups/greenfields
mkdir -p $BACKUP_DIR
pg_dump -U postgres greenfields_audit > $BACKUP_DIR/greenfields_audit_$TIMESTAMP.sql
# Hapus backup lebih dari 30 hari
find $BACKUP_DIR -name "*.sql" -mtime +30 -delete
```

8.2 Restore PostgreSQL

```
# Pilih file backup terbaru
psql -U postgres greenfields_audit < /backups/greenfields/greenfields_audit_20250524_120000.sql
```

Catatan: Restore akan menghapus semua data saat ini dan menggantinya dengan data dari backup.

Pastikan untuk menghentikan aplikasi selama proses restore untuk mencegah inkonsistensi data.

9. Monitoring dan Logging

Akses log untuk monitoring:

```
# Backend logs (real-time)
sudo journalctl -u greenfields-backend -f

# Nginx access & error logs
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log

# PostgreSQL logs
sudo tail -f /var/log/postgresql/postgresql-*.log

# Database active connections
sudo -u postgres psql -c "SELECT * FROM pg_stat_activity WHERE datname = 'greenfields_audit';"
```

```
# Slow queries
```

```
sudo -u postgres psql -c "SELECT query, calls, total_time FROM pg_stat_statements ORDER BY total_time DESC LIMIT 10;"
```

Metric yang direkomendasikan untuk dimonitor:

API response time dan error rate

Database connection usage dan disk usage

Nginx 4xx/5xx response rate

Jumlah active CRITICAL dan HIGH incidents

Status health check endpoint

10. Catatan Keamanan

Ubah password default admin setelah login pertama

Gunakan SECRET_KEY yang kuat (min 32 karakter) dan simpan melalui environment variable

Batasi akses database hanya dari host terpercaya (listen_addresses di postgresql.conf)

Aktifkan HTTPS/TLS untuk production environment

Nonaktifkan atau batasi endpoint /api/auth/seed pada production

Batasi akun admin hanya untuk user operasional yang terpercaya

Gunakan firewall (ufw) untuk membatasi akses port: buka 80, 443, 22; tutup 5432 dan 8000

Set permission file .env menjadi 600 (hanya owner yang bisa membaca)

Aktifkan Nginx rate limiting pada endpoint login

11. Strategi Rollback

Jika terjadi masalah setelah deployment, lakukan rollback dengan langkah berikut:

```
# 1. Simpan artifact build sebelumnya sebelum deployment baru
```

```
# 2. Hentikan backend service versi baru
```

```
sudo systemctl stop greenfields-backend
```

```
# 3. Restore backend versi sebelumnya dari backup
```

```
sudo cp -r /opt/greenfields/backend_bak/* /opt/greenfields/backend/
```

```
# 4. Restore folder frontend dist sebelumnya
```

```
sudo cp -r /opt/greenfields/frontend/dist_bak/* /opt/greenfields/frontend/dist/
```

```
# 5. Restart service
```

```
sudo systemctl restart greenfields-backend
```

```
sudo nginx -s reload
```

```
# 6. Verifikasi
```

```
curl http://localhost/health
```

12. Kesimpulan

Arsitektur deployment Greenfields IMS mendukung production readiness untuk MVP melalui:

Nginx reverse proxy untuk serve frontend dan proxy API

systemd process management untuk auto-restart dan graceful shutdown

PostgreSQL untuk data persistence dengan backup harian

Health check endpoint (/health) untuk monitoring

Backend FastAPI yang stateless (JWT-based, tanpa session server)

Sistem dapat diskalakan secara vertical dengan menambah resource server (CPU/RAM) dan secara horizontal dengan menambahkan backend instance di balik Nginx upstream. Strategi rolling deployment memungkinkan proses release dengan near-zero downtime.

Untuk production penuh, disarankan menambahkan HTTPS/TLS (Certbot/Let's Encrypt), memisahkan database server, dan mengimplementasikan monitoring terpusat (Prometheus/Grafana).